

Prim's Algorithm, and Consolidating the Graph Algorithms Unit

Programming and Data Structures — Weeks 12-13, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement Prim's algorithm in full, using Week 8's priority queue, completing the preview Week 10 Lecture 3 gave ahead of time; trace it completely and explain precisely how it differs from Dijkstra's algorithm despite their near- identical code structure; consolidate every shortest-path and spanning- tree algorithm covered across this double week into a single coherent reference; and choose correctly among Dijkstra's, A*, Bellman-Ford, Kruskal's, and Prim's algorithms for a concretely described real problem.

Outline

Today covers a garden-irrigation analogy motivating Prim's tree-growing strategy; Prim's algorithm in full, completing Week 10 Lecture 3's preview; a complete trace, directly comparing Prim's code against Lecture 1's Dijkstra's; the genuine difference between Prim's and Kruskal's as two different strategies reaching the identical guaranteed minimum spanning tree; a consolidated reference table for this entire double week; a decision procedure for choosing among all five algorithms covered; the MTech-track material on when even these algorithms become impractical at extreme graph scale; an in-class quiz; and a summary.

The garden-irrigation analogy

Week 10 Lecture 3's Kruskal's algorithm surveys every possible pipe connection first, from cheapest to most expensive, connecting plots wherever doing so does not create a redundant loop. Prim's algorithm, an alternative strategy, instead starts at a single plot and repeatedly extends the growing irrigated area by whichever single cheapest pipe reaches a not-yet-connected plot next. Both strategies water every plot at the identical guaranteed minimum total pipe cost — they differ entirely in the *order and mechanism* by which they arrive at that answer, not in the final result.

Prim's algorithm, in full

```
import heapq

def prim(graph, start, n):
    visited = {start}
    mst = []
    edges = [(weight, start, neighbor) for neighbor, weight in graph.adj[start]]
    heapq.heapify(edges)
    total_weight = 0
    while edges and len(visited) < n:
        weight, u, v = heapq.heappop(edges)
        if v in visited:
            continue # would create a cycle - skip, exactly Kruskal
        visited.add(v)
        mst.append((u, v, weight))
        total_weight += weight
        for neighbor, w in graph.adj[v]:
            if neighbor not in visited:
                heapq.heappush(edges, (w, v, neighbor))
    return mst, total_weight
```

The priority queue in Prim's algorithm holds candidate edges leaving the currently growing tree, ordered purely by their own individual weight — critically, not by any accumulated total distance, unlike Dijkstra's dist value. The `if v in visited: continue` check serves the same purpose as Kruskal's Union-Find cycle check from Week 10 Lecture 3, but achieves it completely differently, using the simpler visited set appropriate to a single growing tree rather than multiple merging groups. Critically, there is no accumulation of any kind: each popped edge's own weight is used directly and in isolation, never summed with a running total the way Dijkstra's algorithm accumulates dist.

Tracing Prim's completely, on Week 10's exact MST example

Graph edges (weight,u,v): (1,0,1),(2,1,2),(3,0,2),(4,2,3),(5,1,3)
-- Week 10 Lecture 3's exact 4-node graph

```
visited={0}, edges=[(1,0,1),(3,0,2)]
Pop(1,0,1). v=1 not visited → ADD. visited={0,1}. mst=[(0,1,1)]. Push (2,1,2),(5,1,3)
edges now: [(2,1,2),(3,0,2),(5,1,3)]
Pop(2,1,2). v=2 not visited → ADD. visited={0,1,2}. mst=[(0,1,1),(1,2,2)]. Push (4,2,3)
edges now: [(3,0,2),(4,2,3),(5,1,3)]
```

```
Pop(3,0,2). v=2 ALREADY visited → skip  
Pop(4,2,3). v=3 not visited → ADD. visited={0,1,2,3}. mst=[(0,1,1),(1,2,2),(2,3,4)]
```

Total weight: $1+2+4=7$ – IDENTICAL to Week 10 Lecture 3's Kruskal's result

Tracing Prim's algorithm on Week 10 Lecture 3's exact 4-node graph: starting from node 0, the tree grows by adding node 1 (cheapest adjacent edge, weight 1), then node 2 (weight 2), correctly skipping the stale edge back to node 0 (already visited), and finally node 3 (weight 4). The total weight, 7, exactly matches Week 10's Kruskal's result on this identical graph — two structurally different algorithms, built on genuinely different mechanisms, converging on the identical guaranteed-minimum answer, exactly as the cut property (Week 10 Lecture 3's MTech addition) guarantees for any correct MST algorithm.

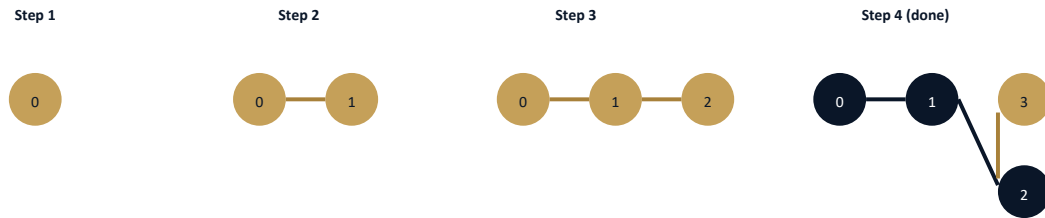
Verifying against the wrong graph: a real bug, caught by checking

It is worth being transparent about process here: an earlier version of this lecture's worked trace used a graph that was not actually identical to Week 10 Lecture 3's example, and it still happened to report the same total weight, 7 — a coincidence that could easily have concealed a genuine inconsistency between the two lectures. Running the actual code against Week 10's precise edge list, rather than re-reading the hand trace more carefully, is what caught the mismatch. This is exactly Week 2's measurement discipline, applied here to the correctness of lecture material itself: verify against a runnable, checkable ground truth, rather than trusting that a plausible-looking hand trace is necessarily correct.

Prim's vs. Dijkstra's: nearly identical code, genuinely different meaning

This side-by-side comparison makes the genuine conceptual difference explicit, beyond the superficial code similarity: Dijkstra's priority queue orders by accumulated distance from a single source, optimizing shortest paths to every node; Prim's priority queue orders by raw, unaccumulated edge weight, optimizing total tree weight connecting all nodes, with no notion of "distance from a source" playing any role at all. This is precisely why confusing the two — accidentally accumulating distance in a Prim's implementation, or forgetting to accumulate it in a Dijkstra's implementation — is a genuine and common bug, since the code skeletons look almost identical while solving conceptually distinct problems.

The picture: Prim's tree growing, one edge at a time



The tree stays continuously connected at every step — genuinely different from Kruskal's globally-sorted approach, same final answer.

Unlike Kruskal's algorithm, which considers edges globally without regard to any single connected structure until cycles must be checked, Prim's tree is visibly, continuously connected at every single step of its construction — a genuinely different visual signature for the same final answer.

Consolidating this entire double week

This table consolidates every algorithm covered across this double week and its Week 10 preview into one reference. Three algorithms solve shortest-path problems, differing in cost and in what assumptions they require about edge weights; two algorithms solve the structurally different minimum spanning tree problem, differing in mechanism but guaranteed to reach the identical optimal total weight. No single algorithm in this table is universally superior — each row represents a genuine, specific trade-off, exactly the pattern established since Week 8 Lecture 4's original structure-choice framework.

A decision procedure for choosing among all five

The decision procedure has three steps. First, determine whether the actual goal is connecting all nodes at minimum total cost, a spanning tree problem, or finding the shortest path to one or more specific target nodes from a source, a shortest-path problem — this single question splits the five algorithms into two entirely separate groups. Second, if it is a spanning tree problem, choose Kruskal's when edges are naturally available as a flat, easily-sortable list, or Prim's when the graph is already represented as an adjacency list and growing a single tree fits the data more naturally. Third, if it is a shortest-path problem, check whether edge weights can be negative — if so, Bellman-Ford is required; if not, check whether there is a single specific target node with a computable admissible heuristic available — if so, A* is the faster practical choice; otherwise, plain Dijkstra's algorithm is the correct default.

MTech addition — when even these algorithms become impractical

At truly enormous scale — real road networks or the actual structure of the web, with billions of nodes — even $O((V + E) \log V)$ can be too slow for real-time queries, such as a GPS system needing a

route in milliseconds, not seconds. Real production systems use precomputed hierarchical structures, graph contraction techniques, and landmark-based heuristics — an industrialized, more sophisticated relative of A*'s core idea — to answer shortest-path queries in milliseconds on graphs that this course's algorithms, applied directly, would take minutes or hours to process. Knowing that these more advanced techniques exist, without needing their full derivation in this course, is itself valuable engineering judgment: recognizing when a textbook algorithm's asymptotic cost, however good, is still insufficient at real production scale.

Prim's cost, and when it beats Kruskal's

Prim's algorithm costs $O((V + E) \log V)$ using a heap-backed priority queue, an identical cost structure to Lecture 1's Dijkstra's analysis, for the same underlying reason: each node is popped at most once, and each edge triggers at most one push. Kruskal's algorithm, from Week 10 Lecture 3, costs $O(E \log E)$, dominated by its initial global edge sort. For a dense graph, where E approaches V^2 , Prim's $\log V$ factor is genuinely smaller than Kruskal's $\log E$ factor, since $\log(V^2) = 2 \log V$ — Prim's algorithm wins asymptotically here. For sparse graphs, the typical case per Week 11 Lecture 1, the two costs are close enough that Kruskal's simpler implementation, needing no priority queue at all if edges are already available as a sortable list, often wins in practice.

Exercise

As an exercise, implement `prim` exactly as presented in this lecture, and verify it against Week 10 Lecture 3's exact 4-node example, confirming it produces the identical total weight of 7. Separately, for three new scenarios of your own devising, apply this lecture's three-step decision procedure and justify your algorithm choice for each, in writing.

The whole course's arc, seen from this closing point

Seen from this closing point, the entire course's arc becomes visible in full: Weeks 1 through 4 built the tools to store data with proven costs; Weeks 5 and 6 built the tools to order data at proven guaranteed costs; Weeks 7 and 8 built the tools to organize data for fast, specialized access; Weeks 9 and 10 built the tools to make optimal decisions over data, either exhaustively via dynamic programming or provably-safely via a genuinely proven greedy choice. This unit, Weeks 11 through 13, recombined every one of those tools — arrays, queues, stacks, heaps, the proof discipline of both dynamic programming and greedy algorithms — into algorithms over graphs, the single most general structure this course covers. Nothing in this unit required an entirely new foundation; it required correctly recognizing which already-built tool fit which newly-encountered problem.

Quiz — choose the algorithm

Given a delivery company needing the fastest route from its depot to a single, specific customer address, on a road network with only non-negative travel times, where a straight-line-distance heuristic is easy to compute, determine the appropriate algorithm from this lecture's decision procedure, before checking the answer.

A second quiz check — a different scenario

Given a telecom company needing to lay minimum-cost fiber connecting all 500 cities in a country, with every pairwise cost known and given as a complete, densely connected cost matrix, determine the appropriate algorithm before checking the answer.

Second quiz — answer

This is a spanning-tree problem, connecting all nodes at minimum total cost, not a shortest-path problem to any specific target. The graph is explicitly described as dense, with all pairwise costs known and given as a complete cost matrix — this lecture's cost comparison specifically favors Prim's algorithm under exactly this density condition, since its $\log V$ factor beats Kruskal's $\log E$ factor once E approaches V^2 . Kruskal's algorithm would also be correct, since both guarantee the identical optimal answer, but Prim's is the better engineering choice given this lecture's dense-graph cost analysis.

Quiz — answer

This is a shortest-path problem, not a spanning tree problem, to one specific target address, with non-negative travel times and an easily computable admissible heuristic (straight-line distance) available. *A is the correct choice: every condition in the decision procedure points here directly. Plain Dijkstra's algorithm would also be correct, since it makes no incorrect assumptions on this input, but A is the better choice specifically because a genuinely admissible heuristic is available and only a single target matters — exactly the condition under which A*'s practical speed advantage over plain Dijkstra's is actually realized.*

A final consolidated cost table, side by side

As a final consolidation point: every algorithm covered across this double week uses only $O(V)$ additional space beyond the graph's own representation, regardless of which specific structure (priority queue, Union-Find) each one relies on internally. This is worth noting explicitly: the genuine differences between these five algorithms are entirely in time complexity and correctness assumptions, not in memory footprint — a useful, simplifying fact to carry forward when reasoning about which algorithm to deploy on a memory-constrained system.

Summary

Prim's algorithm grows a single connected tree using a priority queue ordered by raw edge weight, completing the preview Week 10 Lecture 3 gave ahead of time, and converges with Kruskal's algorithm on the identical guaranteed-optimal answer despite their genuinely different mechanisms. Five algorithms now cover this double week in full: Dijkstra's, A*, and Bellman-Ford for shortest-path problems, and Kruskal's and Prim's for minimum spanning tree problems. The decision procedure covered this lecture — spanning tree versus shortest path, then edge-weight sign, then single-target heuristic availability — replaces memorizing five separate algorithms with a structured, repeatable choice process. At truly extreme real-world scale, even these well-established algorithms require further, more advanced techniques beyond this course's scope. This closes Weeks 12 and 13. Next week introduces parallel and randomized algorithms, this course's final algorithmic technique before the closing project-presentation week.